

SULIT



SECP2523 DATABASE

SESSION 2024/2025 - SEMESTER 1

ALTERNATIVE ASSESSMENT REPORT: PHASE 1

Name	EVELYN GOH YUAN QI
IC No. / Matric No.	041111-13-0600 / A23CS0222
Year / Program	2 / SECPH
Section	2
Lecturer Name	Dr. SEAH CHOON SEN
Case Study	MEMBER REGISTRATION
System Name	Koperasi KADA All-in-one Financial Platform
Group Name	Tech EcoPioneer

Contents

SECTION A: SYSTEM'S OVERVIEW	3
1.1 Overall Description.....	3
1.2 Project Weaknesses or Improvement.....	4
1.3 Proposed Module	5
SECTION B: DATABASE PLANNING AND DESIGN	7
2.1 Conceptual ERD	7
2.2 Logical ERD	8
2.3 Data Dictionary for Logical ERD	9
2.4 Relational Database Schema.....	11
2.5 Normalization	13

SECTION A: SYSTEM'S OVERVIEW

1.1 Overall Description

The Koperasi KADA All-in-One Financial Platform aims to modernize and simplify the cooperative's operations. It reduces inefficiencies created by manual processes, enhances financial transparency and increases access for various members and administrators. The primary goal of the system is to provide a comprehensive, user-friendly solution for savings, loans and other cooperative operations. The key features of the system include login module, member registration module, savings module, loan module and reporting module.

I was assigned to develop Member Registration module. The Member Registration module is a critical component of the cooperative system which aimed to streamline the registration process for new users. It replaces the inefficient manual registration system with an automated, secure and easy to use alternative. The module enables the user to conveniently and faster register online by providing their personal details. It also allows the administrators to easily access and validate the applications with respect to cooperation policies. This would help to promote diversity, reduce paperwork and shortens delays.

The current issues that prompted the initiation of the WBL project is the cooperative's dependance on manual system for processing member registrations, savings, loans and reporting posed considerable issues including:

- **Lack of Integration:** Manual workflows with disjointed processes were limited to efficiency and data uniformity. Administrators need to manually transfer data which increased the chance for errors.
- **Security issues:** Various serious financial and personal information are at danger of compromise because of the lack of good security measures like encryption and multi-factor authentication.
- **Lack of Accessibility:** The existing system required physical presence for tasks such as registration, make simple deposits and apply for loans.
- **Static member engagement:** Members lacked real-time engagement features such as notifications, transaction tracking and user-friendly interfaces leading to delay and dissatisfaction.

Module Distribution Table:

Group members' name	Matric No	Module
Dheshighan A/L Saravana Moorthy	A23CS0072	Login
Evelyn Goh Yuan Qi	A23CS0222	Member Registration
Chew Chiu Xian	A23CS0061	Savings
Ng Yu Hin	A23CS0148	Loan
Lau Yan Kai	A23CS0098	Reporting

1.2 Project Weaknesses or Improvement

The **Member Registration Module** eliminates manual inefficiencies but still has some limitations that could be improved:

1. Validation Bottlenecks:

- Weakness: Manual validation by staff can cause delays when there is a large volume of applications.
- Improvement: Develop a more systematic workflow with task tracking to prioritize applications as well as retain manual validation.

2. Lack of Transparency:

- Weakness: Applicants cannot track the status of their applications, leading to uncertainty and dissatisfaction.
- Improvement: Create a status tracking feature for applicants to check their application status (e.g., Pending, Approved, or Rejected).

1.3 Proposed Module

Module: Member Registration and Management

Module Description

The Member Registration and Management module is a critical enhancement to the cooperative system, offering seamless interactions between users and staff. It builds upon the Member Registration module by addressing validation bottlenecks and improving transparency for applicants.

One of the use cases is Membership Registration. Through an online membership registration form, users can provide their personal details, such as name, phone number, address, and so on. The system validates the data for completeness and correctness before submission. After completely submitting the application, the system stores the application with a status of 'Pending.'

The next use cases are Approve and Reject Membership. Staff members play a key role in this module, using a dedicated interface to review and validate the submitted applications. Based on their evaluation, staff can approve or reject the applications. Notifications of the application status are then sent to users. Once the applications are approved by the staff, members can open and manage accounts for loans and savings.

The enhanced module introduces additional use cases to overcome identified weaknesses, includes:

- **Validation Workflow:** Staff can view and prioritize applications in a queue based on their submission timestamps, ensuring a fair and timely process.
- **Application Status Tracking:** Applicants can check the status of their applications, categorized as Pending, Approved, or Rejected. Notifications of application status changes are automatically sent to users.

Transaction Requirement: Data Entry

The purpose of this transaction is to collect and store member application details and validation tasks. Users can complete the membership registration application by entering their personal details and necessary information. After completing the application, users can submit it. Once submitted, the application details are stored in the database, and the application status is set to 'Pending'. The system adds the application to the validation task queue for staff to review.

Transaction Requirement: Data Update/Deletion

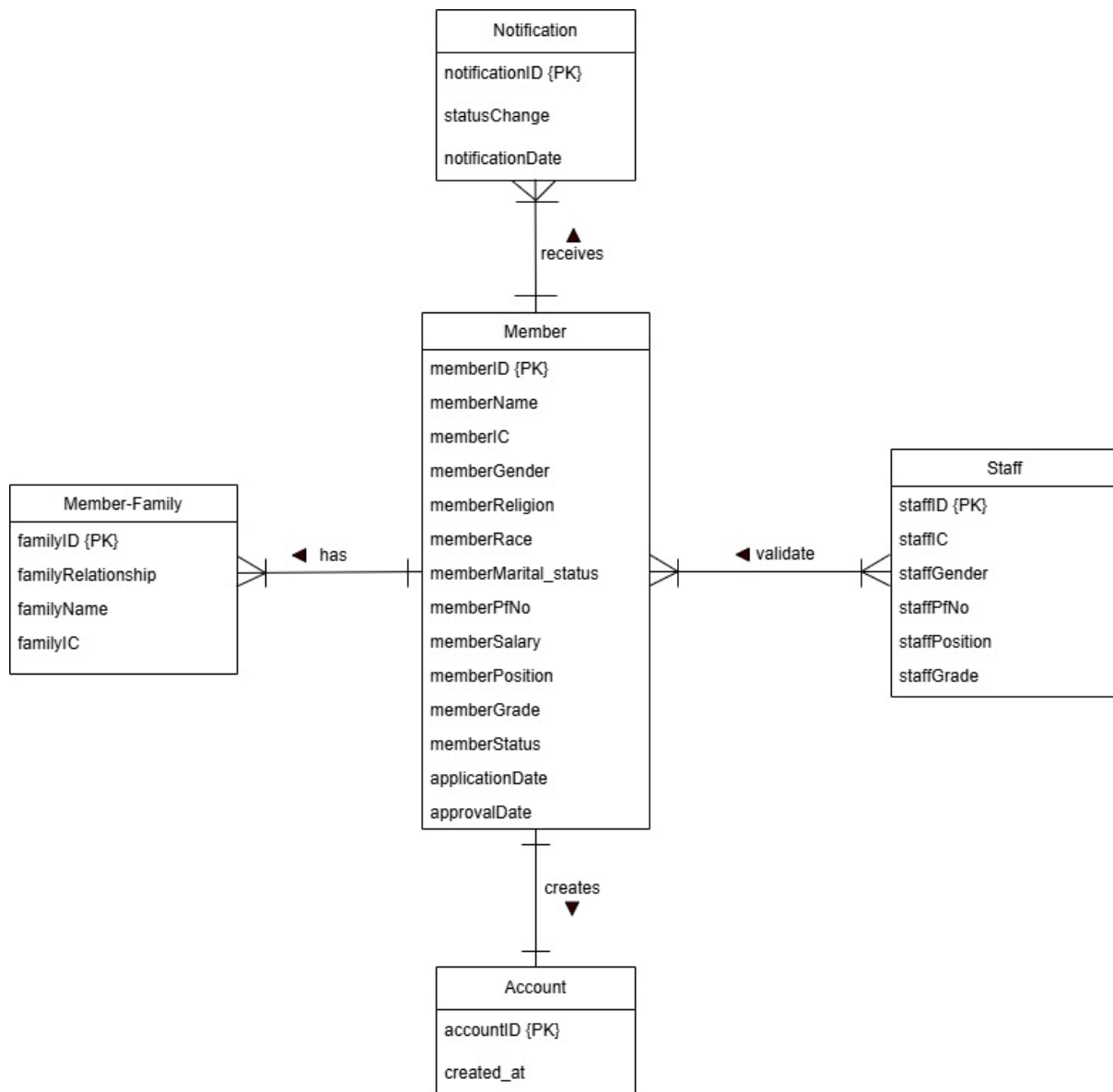
- **Data Update:** Staff can update the status of an application from 'Pending' to either 'Approved' or 'Rejected' during the validation process.
- **Data Deletion:** Staff can delete duplicate or incomplete applications. Rejected applications are archived in the system for record-keeping purposes.

Transaction Requirement: Data Queries

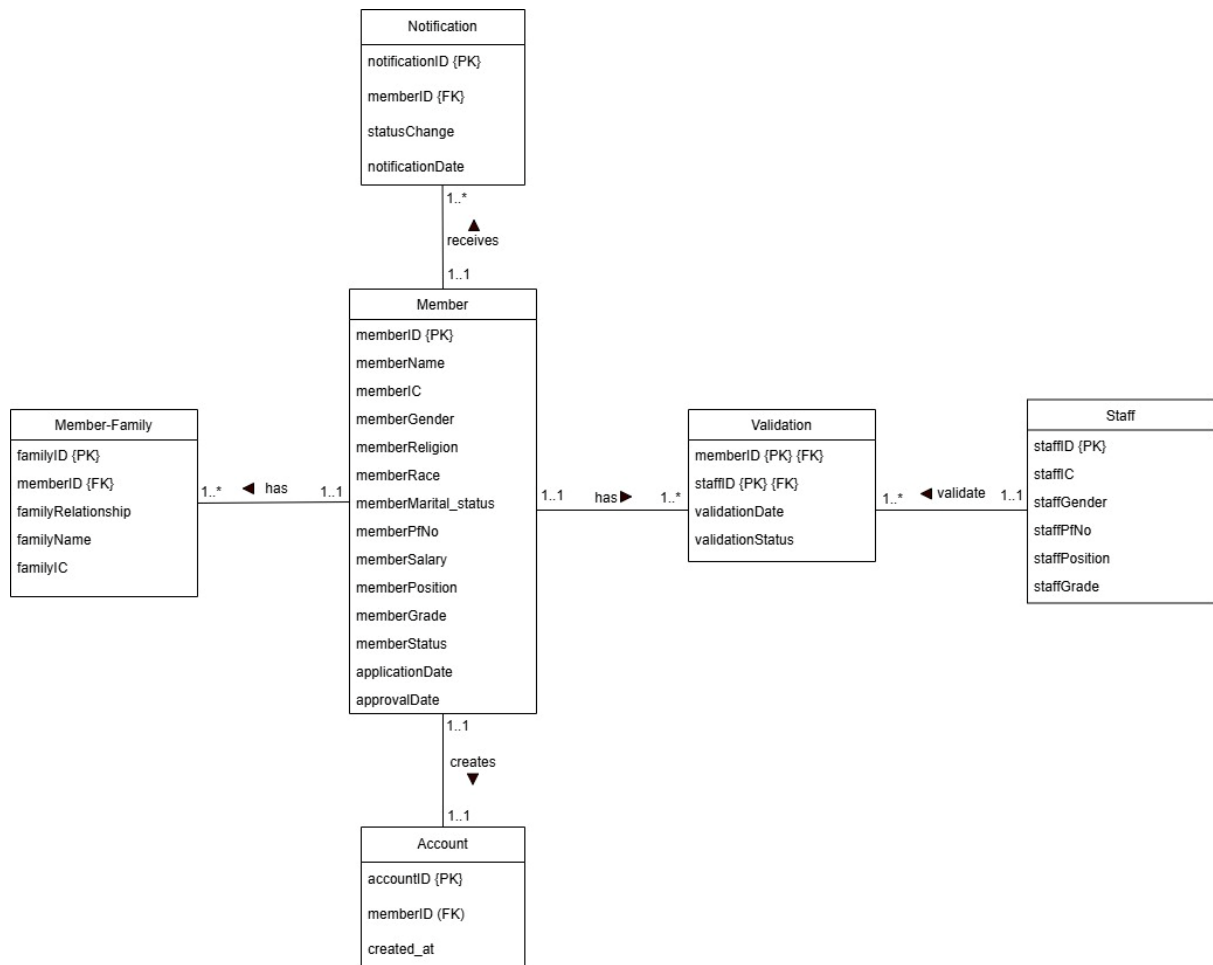
1. The list of pending applications can be retrieved by staff from the validation task queue, prioritized by submission timestamps.
2. The list of approved applications can be retrieved by staff.
3. The list of rejected applications can also be retrieved by staff.

SECTION B: DATABASE PLANNING AND DESIGN

2.1 Conceptual ERD



2.2 Logical ERD

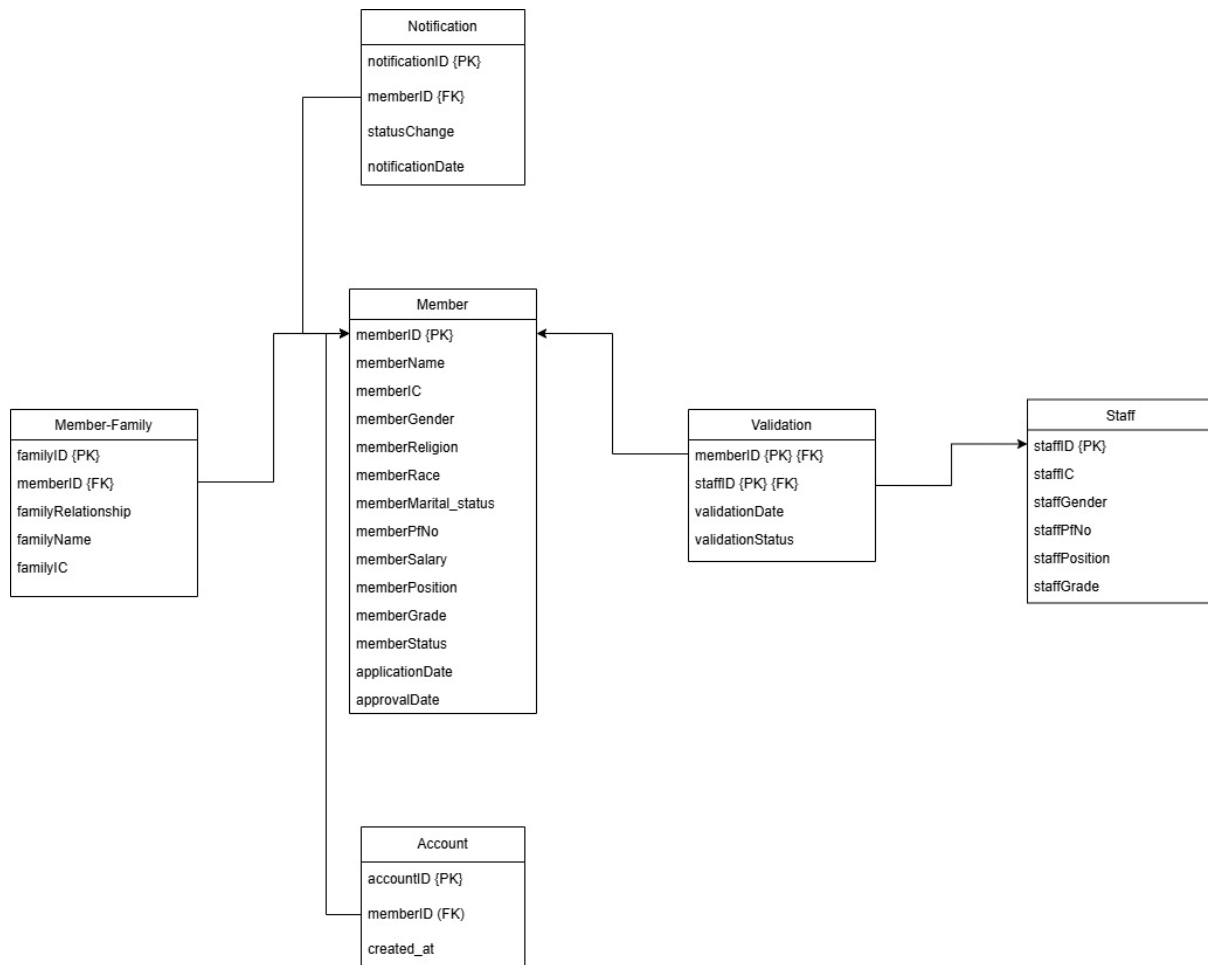


2.3 Data Dictionary for Logical ERD

Entity Name	Attributes	Data type & length	Description
Member	memberID (PK)	int	Unique identifier for each member
	memberName	varchar (255)	Member's name
	memberIC	varchar(22)	Member's IC
	memberGender	enum('Male','Female')	Member's gender
	memberReligion	varchar(50)	Member's religion
	memberRace	varchar(50)	Member's race
	memberMarital_status	enum('Single','Married','Divorced','Widowed')	Member's marital status
	memberPfNo	int	Member's Pf no
	memberSalary	decimal(10,2)	Member's salary
	memberPosition	varchar(100)	Member's position
	memberGrade	varchar(50)	Member's grade
	memberStatus	enum('Pending','Lulus','Tolak')	Member's
	applicationDate	timestamp	Date and time when the member submitted their application
	approvalDate	timestamp	Date and time when the application was approved or rejected
Staff	staffID (PK)	int	Unique identifier for each staff member
	staffIC	varchar(22)	Staff's IC
	staffGender	enum('Male','Female')	Staff's gender
	staffPfNo	int	Staff's Pf no
	staffPosition	varchar(100)	Staff's Position
	staffGrade	varchar(50)	Staff's grade
Validation	memberID (PK) (FK)	int	Foreign key linking the validation to a member
	staffID (PK) (FK)	int	Foreign key linking the validation to a staff
	validationDate	timestamp	Date and time when the validation occurred
	validationStatus	enum('Pending','Lulus','Tolak')	Status of the validation
Member-Family	familyID (PK)	int	Unique identifier for the family record.
	memberID (FK)	int	Foreign key linking the family record to a member
	familyRelationship	varchar (50)	Family member's relationship
	familyName	varchar (100)	Family member's Name
	familyIC	varchar (22)	Family member's IC

Notification	notificationID (PK)	int	Unique identifier for each notification
	memberID (FK)	int	Foreign key linking the notification to a member
	statusChange	varchar (50)	Description of the status change
	notificationDate	timestamp	Date and time when the notification was sent
Account	accountID (PK)	int	Unique identifier for each account
	memberID (FK)	int	Member's ID(refer Member entity)
	created_at	timestamp	Timestamp of when the account was created

2.4 Relational Database Schema



1. Member (**memberID**, memberName, memberIC, memberGender, memberReligion, memberRace, memberMarital_status, memberPfNo, memberSalary, memberPosition, memberGrade, memberStatus, applicationDate, approvalDate)

Primary Key: memberID

2. Staff (**staffID**, staffIC, staffGender, staffPfNo, staffPosition, staffGrade)

Primary Key: staffID

3. Validation (**memberID**, **staffID**, validationDate, validationStatus)

Primary Key: (memberID, staffID)

Foreign Key: memberID → Member(memberID)

Foreign Key: staffID → Staff(staffID)

4. Member_Family (**familyID**, memberID, familyRelationship, familyName, familyIC)

Primary Key: familyID

Foreign Key: memberID → Member(memberID)

5. Notification(**notificationID**, memberID, statusChange, notificationDate)

Primary Key: notificationID

Foreign Key: memberID → Member(memberID)

6. Account (**accountID**, memberID, created_at)

Primary Key: accountID

Foreign Key: memberID → Member(memberID)

2.5 Normalization

1. Member

1NF:

Member (**memberID**, memberName, memberIC, memberGender, memberReligion, memberRace, memberMarital_status, memberPfNo, memberSalary, memberPosition, memberGrade, memberStatus, applicationDate, approvalDate)

Primary Key: memberID

2NF: All attributes are fully dependent on the primary key (memberID). No changes needed since memberID is the only determinant

3NF: No transitive dependency exists

2. Staff

1NF:

Staff (**staffID**, staffIC, staffGender, staffPfNo, staffPosition, staffGrade)

Primary Key: staffID

2NF: No partial dependency exists since staffID is the sole determinant

3NF: No transitive dependency exists

3. Validation

1NF:

Validation (**memberID, staffID**, validationDate, validationStatus)

Primary Key: (memberID, staffID)

Foreign Keys: memberID → Member(memberID), staffID → Staff(staffID)

2NF: All non-key attributes (validationDate, validationStatus) are fully dependent on the composite primary key (memberID, staffID)

3NF: No transitive dependency exists

4. Member-Family

1NF:

Member_Family (**familyID**, memberID, familyRelationship, familyName, familyIC)

Primary Key: familyID

Foreign Key: memberID → Member(memberID)

2NF: All non-key attributes are fully dependent on the primary key (familyID)

3NF: No transitive dependency exists

5. Notification

1NF:

Notification (**notificationID**, memberID, statusChange, notificationDate)

Primary Key: notificationID

Foreign Key: memberID → Member(memberID)

2NF: All non-key attributes are fully dependent on the primary key (notificationID)

3NF: No transitive dependency exists

6. Account

1NF:

Account (**accountID**, memberID, created_at)

Primary Key: accountID

Foreign Key: memberID → Member(memberID)

2NF: All non-key attributes are fully dependent on the primary key (accountID)

3NF: No transitive dependency exists